Web Application Requirements Specification and Analysis App

Sprint Report

<Georgia Papachristou, AM: 4771>

VERSIONS HISTORY

Date	Version	Description	Author
16/5/2024	<1.0>	Full version of the user story requirements.	Georgia Papachristou

1 Introduction

This document provides information concerning the **<4>** sprint of the project.

1.1 Purpose

The purpose of this project is to develop a Web-Based Requirements Specification & Analysis Application designed to streamline and formalize the early stages of the software development lifecycle. The platform enables developers to collaboratively define structured Use Case specifications—complete with actors, preconditions, detailed flows of events, alternative paths, and postconditions—while conducting object-oriented analysis through the creation of CRC (Class-Responsibility-Collaboration) cards. By allowing users to seamlessly establish relations between functional requirements and structural components and automatically compile exportable diagram scripts for visualization tools like PlantUML and Nomnoml, the system successfully bridges the gap between conceptual requirements gathering and technical software design within a secure and intuitive environment.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies the this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project. Finally, Section 4 defines the program architecture using UML diagrams.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Georgia Papachristou
Scrum Master	Georgia Papachristou
Development Team	Georgia Papachristou

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	**1 /4 /2026**	**8 /4 /2026**	**1**	US1, US2, US3
2	**11 /4 / 2026**	**15 /4 / 2026**	**0**	US4, US5, US6
3	**1 /5 / 2026**	**15/ 5/ 2026**	**2**	US7, US8, US9, US10, US11, US12, US13, US14, US15, US16
4	**15/5/2026**	**22/5/2026**	**1**	US18, US19

3 Use Cases

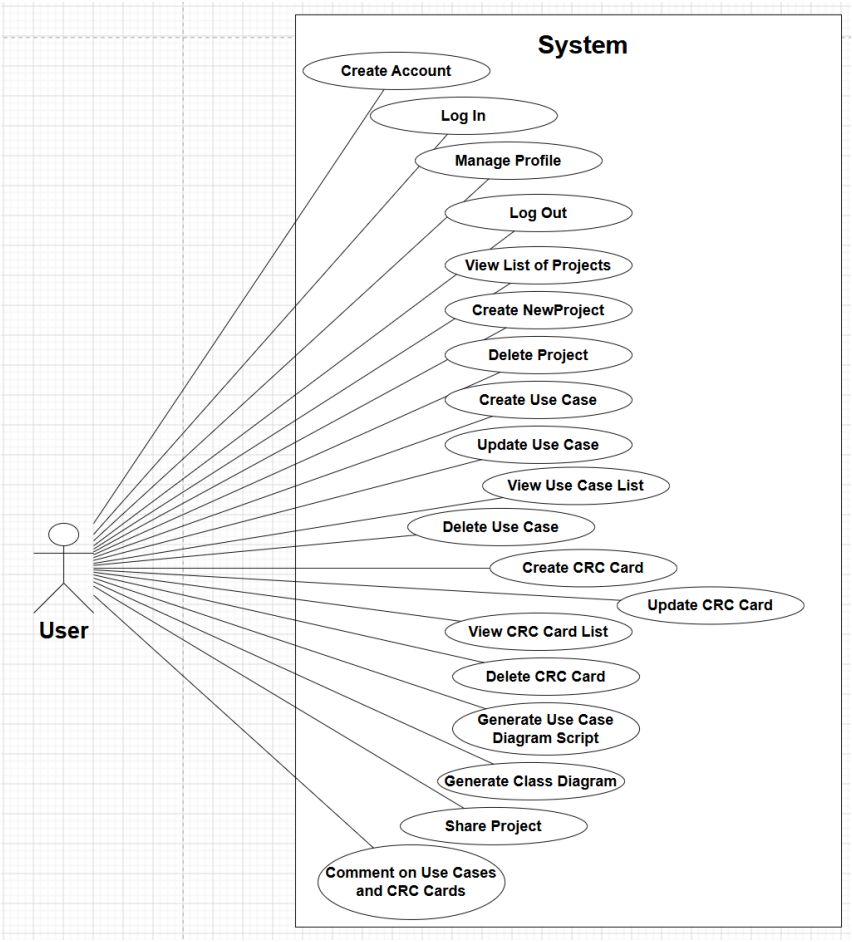

FIGURE 1: USE CASE DIAGRAM

3.1 <Create Account>

Use case ID	UC1
Actors	User
Pre conditions	The user is not authenticated.
Main flow of events	1. The Use Case starts when the User opens the application and navigates the registration page. 2. The User fills in the form: First Name, Last Name, Email, Username, Password. 3. The User clicks the “Create account” button. 4. The system validates that all fields are present and that the username is not already taken. 5. The system encodes the password, assigns the USER role, and persists the new account. 6. The system redirects to the login page and displays a success confirmation message.
Alternative flow 1	If the username is already taken, then the system stays on the registration page and displays an error message.
Alternative flow 2	If the User didn’t fill a required field, then the system validation prevents form submission.
Post conditions	A new User record is persisted in the database.

3.2 <Log In>

Use case ID	UC2
Actors	User
Pre conditions	The user is authenticated.
Main flow of events	1. The Use Case starts when the User enters username and password on the login page and clicks “Sign in”. 2. Spring Security validates the credentials. 3. On success, the system redirects the developer to the personal dashboard
Alternative flow 1	If the User enter Invalid credentials, then the system redirects to login page.
Post conditions	The user has successfully login to his/her account.

3.3 <Manage Profile>

Use case ID	UC3
Actors	User
Pre conditions	The User is logged in and navigates to the profile page.
Main flow of events	1. The Use Case starts when the system loads the profile form with the current user data. 2. The User updates the desired fields and re-enters the password. 3. The User clicks “Save changes”. 4. The system encodes the password and persists in all updated fields. 5. The system displays a success message.
Alternative flow 1	If the User leave blank a required field, then the system displays a validation error.
Post conditions	The User’s profile information is updated in the database

3.4 <Log Out>

Use case ID	UC4
Actors	User
Pre conditions	The User is logged in.
Main flow of events	1. The Use Case starts when the User clicks “Logout”. 2. The system with Spring Security invalidates the current session. 3. The system redirects to the homepage.
Post conditions	The session is terminated.

3.5 <View List of Projects>

Use case ID	UC5
Actors	User
Pre conditions	The User is logged in.
Main flow of events	1. The Use Case starts when the user navigates to the project list page. 2. The system retrieves all the projects associated with the User. 3. The User can see all the projects associated with him. 4. If other Users have shared projects with the current User: 4.1. The system displays shared projects in a separate section in the project page.
Alternative flow 1	If the User doesn’t have projects, then the system displays an empty state message.
Post conditions	No change in system state.

3.6 <Create New Project>

Use case ID	UC6
Actors	User
Pre conditions	The User is logged in.
Main flow of events	1. The Use Case starts when the User clicks “New project” in the project list page. 2. The system displays the project creation form. 3. The User fills in the project information. 4. The User clicks “Create Project”. 5. The system associates the project with the current User. 6. The system persists the project.
Alternative flow 1	Empty project name: Validation error shown.
Post conditions	A new project record has persisted in the Database.

3.7 <Delete Project>

Use case ID	UC7
Actors	User
Pre conditions	The User is logged in and has at least one project.
Main flow of events	1. The Use Case starts when the User clicks “Delete” in a project in the project list page. 2. The system displays a confirmation dialog. 3. The User confirms deletion. 4. The system removes the project.
Alternative flow 1	If the user cancel the deletion in the confirmation dialog, then the system doesn’t delete the project.
Post conditions	The project and all associated content are permanently deleted.

3.8 <Create Use Case>

Use case ID	UC8
Actors	User
Pre conditions	The User is logged in and has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “New Use Case” in the use case list page. 2. The system displays the use case form. 3. The User fills in all required information. 4. The User submits the form. 5. The system persists the use case and the actors entities.
Alternative flow 1	If the User leave empty the use case name, then the system displays a validation error.
Post conditions	A new use case record is persisted.

3.9 <Update Use Case>

Use case ID	UC9
Actors	User
Pre conditions	The User is viewing the use cases list.
Main flow of events	1. The Use Case starts when the User clicks “Edit” in the use case list page. 2. The system displays the populated form. 3. The User modifies fields. 4. The User saves changes. 5. The system updates the use case.
Alternative flow 1	If the User leave empty the use case name, then the system displays a validation error.
Post conditions	The use case is updated.

3.10 <View Use Case List>

Use case ID	UC10
Actors	User
Pre conditions	The User is authorized for the selected project.
Main flow of events	1. The Use Case starts when the User selects “Use Cases” in the project list page. 2. The system retrieves all use cases for the specified project. 3. The system displays them in a list.
Alternative flow 1	If the user doesn’t have created any use case, then the system displays nothing.
Post conditions	No change in system state.

3.11 <Delete Use case>

Use case ID	UC11
Actors	User
Pre conditions	The User is viewing the use cases list.
Main flow of events	1. The Use Case starts when the User clicks “Delete” on a specified use case in the use case list. 2. The system displays a Confirmation dialog. 3. The User confirms deletion. 4. The system deletes the Use Case and all the associated comments.
Alternative flow 1	If the user cancel the deletion in the confirmation dialog, then the system doesn’t delete the use case.
Post conditions	The use case and all associated content are permanently deleted.

3.12 <Create CRC Card>

Use case ID	UC12
Actors	User
Pre conditions	The User is logged in and has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “New CRC Card” in the CRC Cards list. 2. The system displays the CRC Card form. 3. The User fills in the form. 4. The User links this new crc cards with the existing use cases. 5. The User submits the form. 6. The system persists the CRC card.
Alternative flow 1	If the User leave empty the class name, then the system displays a validation error.
Post conditions	A new CRC card is created and persisted.

3.13 <Update CRC Card>

Use case ID	UC13
Actors	User
Pre conditions	The User is logged in and has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “Edit” in a specified CRC Card in the CRC Cards list. 2. The system displays the populated form. 3. The User modifies the information. 4. The User clicks “Save Changes”. 5. The system updates the CRC Card.

Alternative flow 1	If the User leave empty the class name, then the system displays a validation error.
Post conditions	The CRC card is updated.

3.14 <View CRC Card List>

Use case ID	UC14
Actors	User
Pre conditions	The User is logged in and has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “CRC Cards” in a specified project in the project list page. 2. The system retrieves all crc cards for the specified project. 3. The system displays them in a list.
Alternative flow 1	If the user doesn’t have created any crc cards, then the system displays nothing.
Post conditions	No change in system state.

3.15 <Delete CRC Card>

Use case ID	UC15
Actors	User
Pre conditions	The User is logged in and has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “Delete” on a specified crc card in the crc card list. 2. The system displays a Confirmation dialog. 3. The User confirms deletion. 4. The system deletes the crc cards and all the associated comments.

Alternative flow 1	If the user cancel the deletion in the confirmation dialog, then the system doesn't delete the crc card.
Post conditions	The crc card and all associated content are permanently deleted.

3.16 <Generate Use Case Diagram Script>

Use case ID	UC16
Actors	User
Pre conditions	The User is logged in and has access to the selected project with at least one use case and at least one actor.
Main flow of events	1. The Use Case starts when the User clicks "Diagrams" on a specified project in the project list page. 2. The system navigates to the diagrams page. 3. The system displays a form containing "Diagram Type dropdown" and "UML Tool dropdown". 4. The User selects "Use Case Diagram" and a diagram tool ("PlantUML" or Nomnoml). 5. The User clicks "Generate Script". 6. The system displays the generated script inside a read-only textarea. 7. The User may copy the generated script using the **"**Copy to Clipboard**"** button.
Alternative flow 1	If the project has not any use case, then the system displays an error message inside the read-only textarea.
Post conditions	No change in system state.

3.17 <Generate Class Diagram Script>

Use case ID	UC17
Actors	User
Pre conditions	The User is logged in and has access to the selected project with at least one crc card.
Main flow of events	1. The Use Case starts when the User clicks “Diagrams” on a specified project in the project list page. 2. The system navigates to the diagrams page. 3. The system displays a form containing “Diagram Type dropdown” and “UML Tool dropdown”. 4. The User selects “Class Diagram” and a diagram tool (“PlantUML” or Nomnoml). 5. The User clicks “Generate Script”. 6. The system displays the generated script inside a read-only textarea. 7. The User may copy the generated script using the **“**Copy to Clipboard**”** button.
Alternative flow 1	If the project has not any crc cards, then the system displays an error message inside the read-only textarea.
Post conditions	No change in system state.

3.18 <Share Project>

Use case ID	UC18
Actors	User
Pre conditions	The User has access to the selected project and the teammate has an existing account int the system.
Main flow of events	1. The Use Case starts when the User clicks “Share” button on a project list page. 2. The system navigates to the share page. 3. The system displays the current collaborators list and “Add collaborator” form. 4. The User enters the collaborator’s username.

	5. The User clicks “Share Project”. 6. The system adds the collaborator to the collaborators list. 7. The system displays the shared project in the collaborator’s “Shared with me” section.
Alternative flow 1	If the username does not exist, the system displays an error message "No user found with username: <username>".
Alternative flow 2	If the owner (User) attempts to share with themselves, then the system displays an error message "You cannot share a project with yourself."
Alternative flow 3	If the project is already shared to the specified user, then the system displays the message "Project is already shared with <username>".
Alternative flow 4	If the owner (User) clicks “Remove” next to a collaborator, then the system removes the collaborator’s access.
Post conditions	Collaborators can access project resources.

3.19 <Comment on Use Cases and CRC Cards>

Use case ID	UC19
Actors	User
Pre conditions	The User has access to the selected project.
Main flow of events	1. The Use Case starts when the User clicks “Comment” button on a use case or crc card. 2. The system navigates to the comment page. 3. The system retrieves and displays all existing comments ordered by creation time. 4. The User types a new comment in the text area. 5. The User clicks “Post Comment”. 6. The system saves the comment in the database. 7. The comments page reloads and displays the new comment.
Alternative flow 1	If the user submit an empty comment, then the system displays a message “Please fill out this field”.
Post conditions	The comment becomes visible to all authorized collaborators.

4 Design

4.1 Architecture

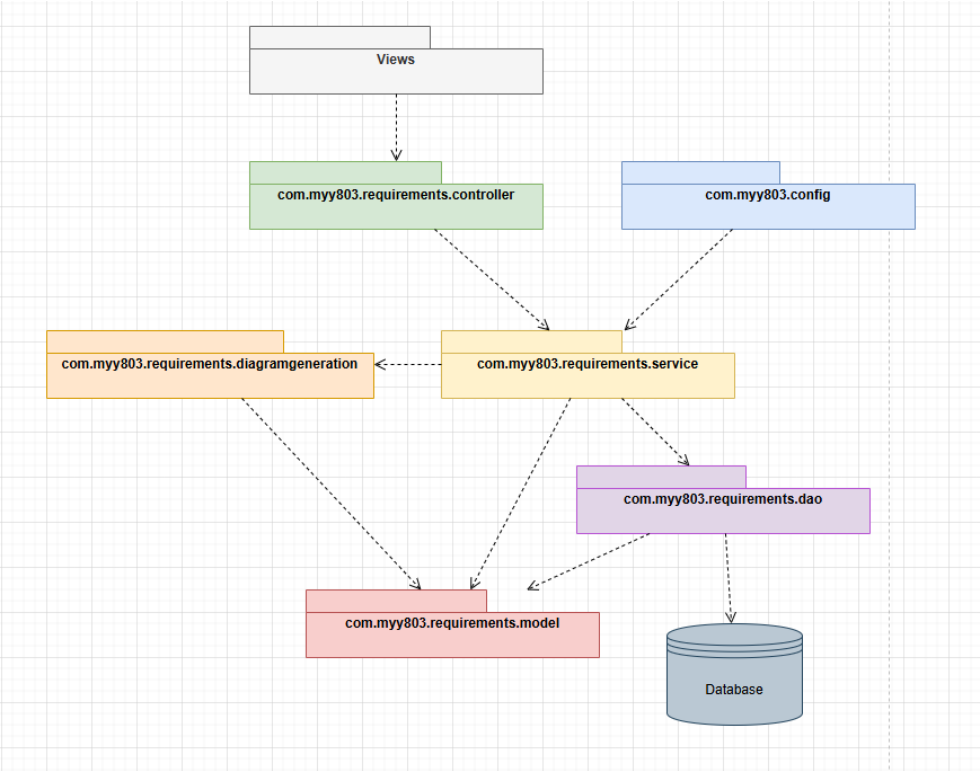

FIGURE 2: PACKAGE DIAGRAM

4.2 Design

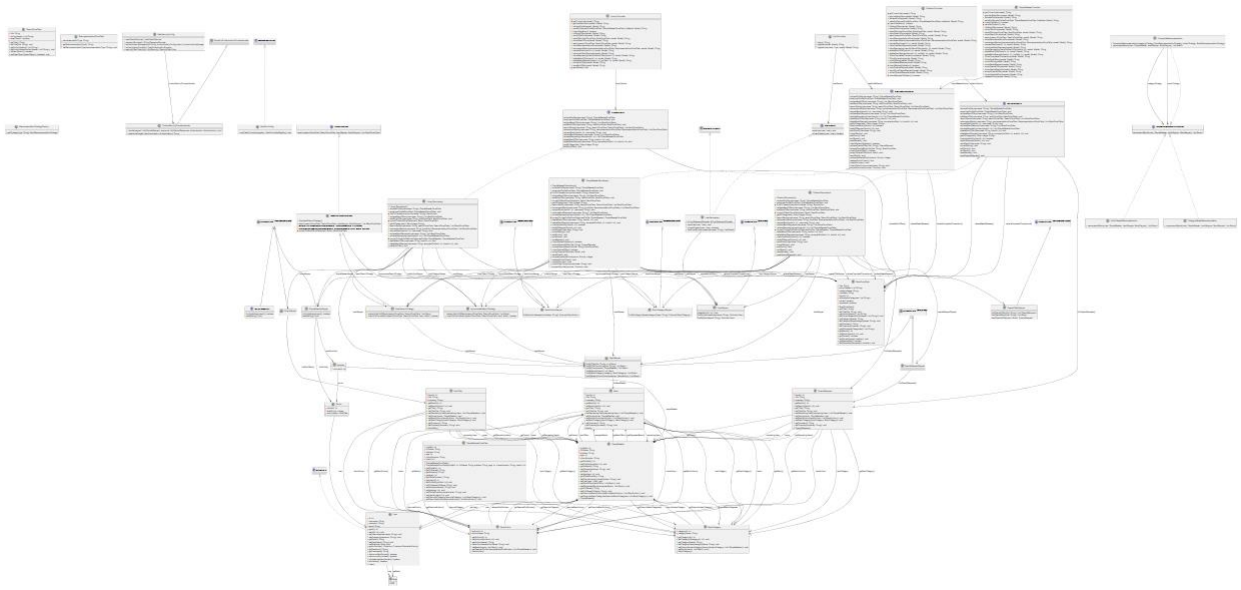

FIGURE 3: ENTIRE UML DIAGRAM

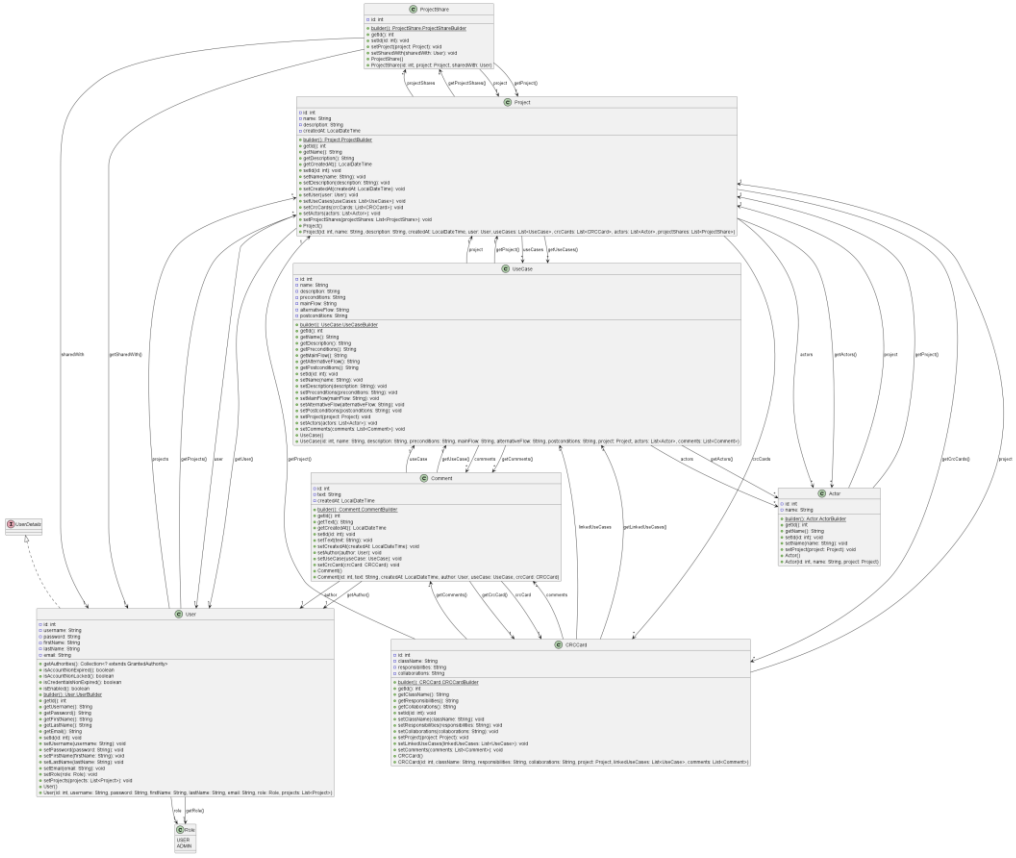

FIGURE 4: MODEL UML DIAGRAM

Class Name: User	
Responsibilities: ▪ Represents system users ▪ Stores authentication data ▪ Stores user profile information ▪ Stores user roles ▪ Stores owned/shared projects	**Collaborations:** ▪ Role ▪ Project ▪ ProjectShare

Class Name: Role	
Responsibilities: ▪ Represents user roles	**Collaborations:** ▪ User

Class Name: Project	
Responsibilities: ▪ Represents projects ▪ Stores project information ▪ Stores project relationships ▪ Links use cases and CRC cards ▪ Supports collaboration/sharing	**Collaborations:** ▪ UseCase ▪ CRCCard ▪ ProjectShare ▪ User

Class Name: UseCase	
Responsibilities: ▪ Represents use cases ▪ Stores actors ▪ Stores descriptions/scenarios	**Collaborations:** ▪ Actor ▪ Comment ▪ Project

Class Name: CRCCard	
Responsibilities: ▪ Represents CRC cards ▪ Stores class responsibilities ▪ Stores collaborators ▪ Associates CRC cards with projects	**Collaborations:** ▪ Comment ▪ Project

Class Name: Comment	
Responsibilities: ▪ Represents comments ▪ Stores comment text ▪ Stores comment metadata ▪ Associates comments with use cases or CRC cards	**Collaborations:** ▪ UseCase ▪ CRCCard ▪ User

Class Name: Actor	
Responsibilities: ▪ Represents use case actors ▪ Stores actor names/descriptions ▪ Associates actors with use cases	**Collaborations:** ▪ UseCase

Class Name: ProjectShare	
Responsibilities: ▪ Represents project sharing relationships ▪ Connects projects and users ▪ Stores collaboration permissions	**Collaborations:** ▪ User ▪ Project

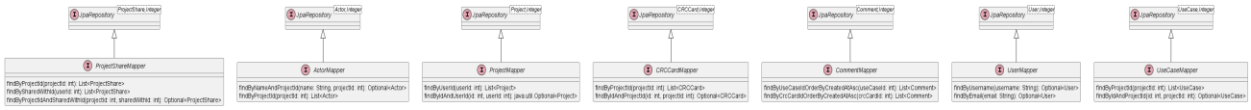

FIGURE 5: DAO UML DIAGRAM

Class Name: UserMapper	
Responsibilities: ▪ Retrieves user records ▪ Saves user records ▪ Updates user records ▪ Deletes user records	**Collaborations:** ▪ User ▪ Database

Class Name: ProjectMapper	
Responsibilities: ▪ Retrieves project data ▪ Saves projects ▪ Updates projects ▪ Deletes projects	**Collaborations:** ▪ User ▪ Database

Class Name: CRCCardMapper	
Responsibilities: ▪ Retrieves CRC card data ▪ Saves CRC cards ▪ Updates CRC cards ▪ Deletes CRC cards	**Collaborations:** ▪ CRCCard ▪ Database

Class Name: UseCaseMapper	
Responsibilities: ▪ Retrieves use case data ▪ Saves use cases ▪ Updates use cases ▪ Deletes use cases	**Collaborations:** ▪ UseCase ▪ Database

Class Name: CommentMapper	
Responsibilities: ▪ Retrieves comments ▪ Saves comments ▪ Updates comments ▪ Deletes comments	**Collaborations:** ▪ Comment ▪ Database

Class Name: ActorMapper	
Responsibilities: ▪ Retrieves actors ▪ Saves actors ▪ Updates actors ▪ Deletes actors	**Collaborations:** ▪ Actor ▪ Database

Class Name: ProjectShareMapper	
Responsibilities: ▪ Retrieves sharing relationships ▪ Saves sharing information ▪ Deletes sharing relationships	**Collaborations:** ▪ ProjectShare ▪ Database

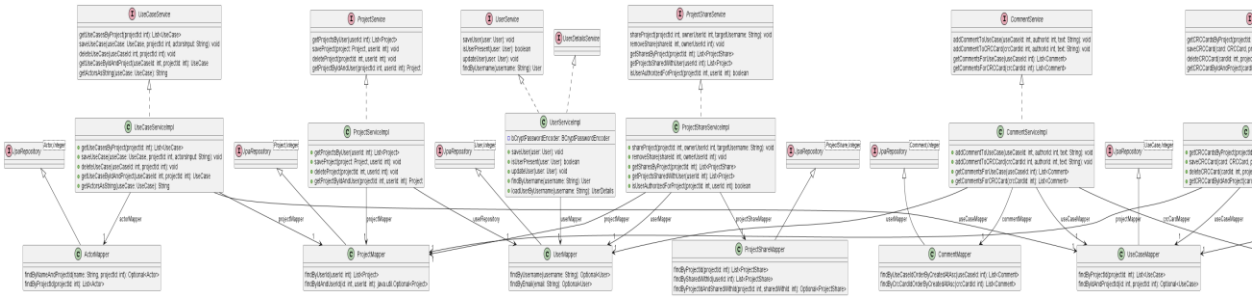

FIGURE 6:SERVICE & DAO UML DIAGRAM

Class Name: UserServiceImpl	
Responsibilities: ▪ Implements user business logic ▪ Loads users for authentication ▪ Encodes passwords ▪ Saves users ▪ Retrieves users ▪ Integrates with Spring Security	**Collaborations:** ▪ UserMapper ▪ Role ▪ User

Class Name: ProjectServiceImpl	
Responsibilities: ▪ Implements project business logic ▪ Saves projects ▪ Updates projects ▪ Deletes projects ▪ Retrieves projects ▪ Handles ownership logic	**Collaborations:** ▪ ProjectMapper ▪ Project ▪ User

Class Name: UseCaseServiceImpl	
Responsibilities: ▪ Implements use case business logic ▪ Saves use cases ▪ Updates use cases ▪ Deletes use cases ▪ Retrieves use cases ▪ Associates actors/comments	**Collaborations:** ▪ UseCaseMapper ▪ ActorMapper ▪ CommentMapper ▪ UseCase ▪ Actor ▪ Comment

Class Name: CRCCardServiceImpl	
Responsibilities: ▪ Implements CRC card business logic ▪ Saves CRC cards ▪ Updates CRC cards ▪ Deletes CRC cards ▪ Retrieves CRC cards ▪ Handles CRC card comments	**Collaborations:** ▪ CRCCardMapper ▪ CommentMapper ▪ CRCCard ▪ Comment

Class Name: CommentServiceImpl	
Responsibilities: ▪ Implements comment business logic ▪ Saves comments ▪ Deletes comments ▪ Retrieves comments ▪ Associates comments with entities	**Collaborations:** ▪ CommentMapper ▪ UseCaseMapper ▪ CRCCardMapper ▪ Comment ▪ UseCase ▪ CRCCard

Class Name: ProjectShareServiceImpl	
Responsibilities: ▪ Implements project sharing logic ▪ Shares projects between users ▪ Removes sharing relationships ▪ Retrieves shared projects	**Collaborations:** ▪ ProjectShareMapper ▪ ProjectMapper ▪ UserMapper ▪ ProjectShare ▪ Project ▪ User

Class Name: DiagramServiceImpl	
Responsibilities: ▪ Implements UML generation logic ▪ Selects diagram generators ▪ Coordinates diagram creation ▪ Generates class diagrams ▪ Generates use case diagrams	**Collaborations:** ▪ ClassDiagramFactory ▪ UseCaseDiagramFactory ▪ ClassDiagramStrategy ▪ UseCaseDiagramStrategy

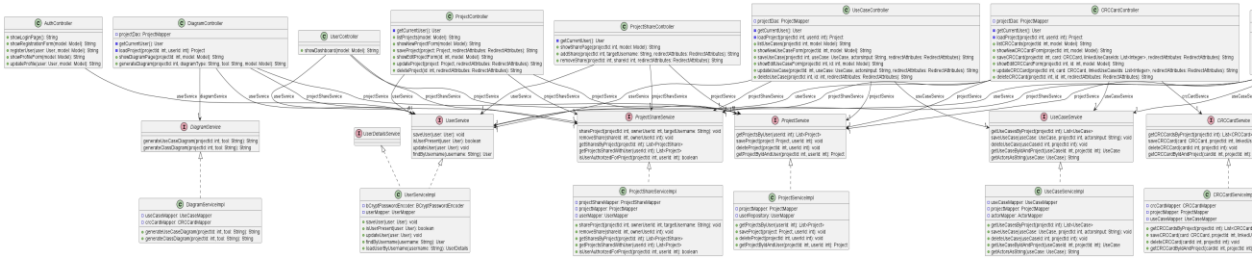

FIGURE 7: CONTROLLER & SERVICE UML DIAGRAM

Class Name: AuthController	
Responsibilities: ▪ Displays login page ▪ Displays registration page ▪ Handles user registration ▪ Validates registration requests ▪ Saves new users	**Collaborations:** ▪ UserService

Class Name: UserController	
Responsibilities: ▪ Displays user dashboard ▪ Handles user-related pages ▪ Loads authenticated user information	**Collaborations:** ▪ UserService

Class Name: ProjectController	
Responsibilities: ▪ Displays project list ▪ Displays project forms ▪ Manages project navigation	**Collaborations:** ▪ ProjectService

Class Name: ProjectShareController	
Responsibilities: ▪ Displays sharing interface ▪ Handles collaboration requests	**Collaborations:** ▪ ProjectShareService ▪ ProjectService ▪ UserService

Class Name: UseCaseController	
Responsibilities: ▪ Displays use cases ▪ Displays use case forms ▪ Associates use cases with projects	**Collaborations:** ▪ UseCaseService ▪ ProjectService

Class Name: CRCCardController	
Responsibilities: ▪ Displays CRC cards ▪ Displays CRC card forms ▪ Associates CRC cards with projects	**Collaborations:** ▪ CRCCardService ▪ ProjectService

Class Name: CommentController	
Responsibilities: ▪ Displays comments ▪ Manages comment forms	**Collaborations:** ▪ CommentService ▪ UseCaseService ▪ CRCCardService

Class Name: DiagramController	
Responsibilities: ▪ Displays diagram generation page ▪ Handles diagram generation requests ▪ Sends generated diagrams to views	**Collaborations:** ▪ DiagramService ▪ ClassDiagramFactory ▪ UseCaseDiagramFactory ▪ Diagram generator strategies

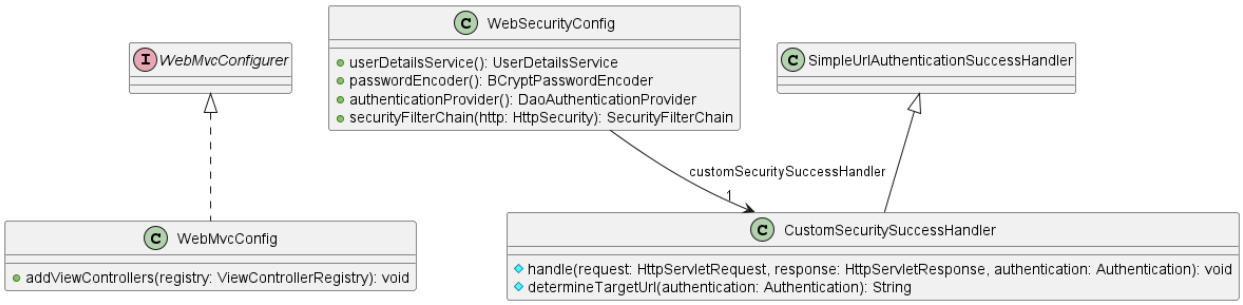

FIGURE 8: CONFIG UML DIAGRAM

Class Name: WebSecurityConfig	
Responsibilities: ▪ Configures Spring Security ▪ Defines authentication rules ▪ Defines authorization rules ▪ Configures login/logout behavior ▪ Configures password encoding ▪ Registers authentication providers	**Collaborations:** ▪ Spring Security ▪ CustomSecuritySuccessHandler ▪ UserServiceImpl ▪ PasswordEncoderstrategies

Class Name: CustomSecuritySuccessHandler	
Responsibilities: ▪ Handles successful authentication events ▪ Redirects users after login ▪ Determines post-login navigation flow	**Collaborations:** ▪ Spring Security

Class Name: WebMvcConfig	
Responsibilities: ▪ Configures Spring MVC ▪ Registers view controllers ▪ Configures static resources ▪ Configures application routing	**Collaborations:** ▪ Spring MVC ▪ Thymeleaf

FIGURE 9: DIAGRAM GENERATION UML DIAGRAM

Class Name: AbstractClassDiagramGenerator	
Responsibilities: ▪ Implements the template method for generating class diagram scripts. ▪ Coordinates the overall generation process for class diagrams. ▪ Generates headers, class blocks, associations, and footers through abstract methods. ▪ Provides shared helper methods for parsing collaborations and responsibilities.	**Collaborations:** ▪ ClassDiagramStrategy ▪ CRCCard ▪ PlantUMLClassDiagramGenerator ▪ NomnomlClassDiagramGenerator

Class Name: AbstractUseCaseDiagramGenerator	
Responsibilities: ▪ Implements the template method for generating use case diagram scripts. ▪ Coordinates the overall use case diagram generation workflow. ▪ Generates headers, actors, use cases, associations, and footers through abstract methods. ▪ Provides reusable helper functionality for subclasses.	**Collaborations:** ▪ UseCaseDiagramStrategy ▪ UseCase ▪ Actor ▪ PlantUMLUseCaseDiagramGenerator ▪ NomnomlUseCaseDiagramGenerator

Class Name: ClassDiagramFactory	
Responsibilities: ▪ Creates the appropriate class diagram generator based on the selected UML tool. ▪ Returns concrete implementations of ClassDiagramStrategy.	**Collaborations:** ▪ ClassDiagramStrategy ▪ PlantUMLClassDiagramGenerator ▪ NomnomlClassDiagramGenerator

Class Name: ClassDiagramStrategy	
Responsibilities: ▪ Defines the common interface for class diagram generation strategies. ▪ Declares the generateScript operation for creating diagram scripts from CRC cards.	**Collaborations:** ▪ AbstractClassDiagramGenerator ▪ PlantUMLClassDiagramGenerator ▪ NomnomlClassDiagramGenerator ▪ CRCCard

Class Name: NomnomlClassDiagramGenerator	
Responsibilities: ▪ Generates class diagram scripts using Nomnoml syntax. ▪ Produces Nomnoml headers and class nodes. ▪ Formats responsibilities using Nomnoml member separators. ▪ Generates association lines between classes using Nomnoml notation. ▪ Produces Nomnoml-compatible output.	**Collaborations:** ▪ AbstractClassDiagramGenerator ▪ CRCCard

Class Name: NomnomlUseCaseDiagramGenerator	
Responsibilities: ▪ Generates use case diagram scripts using Nomnoml syntax. ▪ Produces actor declarations in	**Collaborations:** ▪ AbstractUseCaseDiagramGenerator ▪ UseCase

Nomnoml format. ▪ Produces use case declarations in Nomnoml format. ▪ Generates actor-to-use-case association lines. ▪ Produces Nomnoml-compatible use case diagram output.	▪ Actor

Class Name: PlantUMLClassDiagramGenerator	
Responsibilities: ▪ Generates class diagram scripts using PlantUML syntax. ▪ Produces PlantUML headers and footers. ▪ Generates PlantUML class blocks from CRC cards. ▪ Converts responsibilities into class members. ▪ Generates class association lines using PlantUML notation.	**Collaborations:** ▪ AbstractClassDiagramGenerator ▪ CRCCard

Class Name: PlantUMLUseCaseDiagramGenerator	
Responsibilities: ▪ Generates use case diagram scripts using PlantUML syntax. ▪ Produces PlantUML actor declarations with aliases. ▪ Generates use case declarations inside a system boundary. ▪ Creates associations between actors and use cases.	**Collaborations:** ▪ AbstractUseCaseDiagramGenerator ▪ UseCase ▪ Actor

Class Name: UseCaseDiagramFactory	
Responsibilities: ▪ Creates the appropriate use case diagram generator according to the selected UML tool. ▪ Returns concrete implementations of UseCaseDiagramStrategy.	**Collaborations:** ▪ UseCaseDiagramStrategy ▪ PlantUMLUseCaseDiagramGenerator ▪ NomnomlUseCaseDiagramGenerator

Class Name: UseCaseDiagramStrategy	
Responsibilities: ▪ Defines the common interface for use case diagram generation strategies. ▪ Declares the generateScript operation for creating use case diagram scripts.	**Collaborations:** ▪ AbstractUseCaseDiagramGenerator ▪ PlantUMLUseCaseDiagramGenerator ▪ NomnomlUseCaseDiagramGenerator ▪ UseCase